

# SIMATS ENGINEERING

## ASSESSMENT TOOL 3

**COURSE NAME:** Database Management Systems

**COURSE CODE:** CSA05

### COURSE OUTCOME COVERED

**CO4:** Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)

**Activity Tool :** Comparative Analysis (Low)

**Assessment Tool Weightage:** 10%

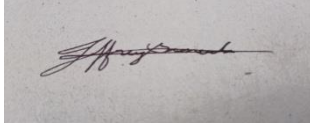
### QUESTIONS

| Q.No | Question                                                                                                                        | Bloom's Level  |
|------|---------------------------------------------------------------------------------------------------------------------------------|----------------|
| 1    | Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency. | BL4 – Analyze  |
| 2    | Differentiate between clustered indexing and non-clustered indexing with suitable database examples.                            | BL4 – Analyze  |
| 3    | Compare primary indexing and secondary indexing based on storage requirements and search performance.                           | BL4 – Analyze  |
| 4    | Analyze the differences between static hashing and dynamic hashing in database systems.                                         | BL4 – Analyze  |
| 5    | Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.                                             | BL4 – Analyze  |
| 6    | Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.                              | BL4 – Analyze  |
| 7    | Compare dense indexing and sparse indexing with respect to memory usage and access speed.                                       | BL4 – Analyze  |
| 8    | Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.                 | BL4 – Analyze  |
| 9    | Compare single-level indexing and multi-level indexing for efficient database retrieval operations.                             | BL4 – Analyze  |
| 10   | Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.          | BL5 – Evaluate |

**Code of Conduct:**

I Jeffrey Samuels.E certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

**RUBRICS FOR CALCULATION:**

| Criteria                | Weightage (%) | Level 4 (Exemplary)                                                           | Level 3 (Proficient)                            | Level 2 (Developing)                       | Level 1 (Beginning)                      |
|-------------------------|---------------|-------------------------------------------------------------------------------|-------------------------------------------------|--------------------------------------------|------------------------------------------|
| Scope of Items          | 20%           | Selects highly relevant items with clear scope and justification              | Selects appropriate items with minor gaps       | Selection is limited or partially relevant | Items are unclear or not relevant        |
| Comparison Depth        | 25%           | Provides detailed and comprehensive comparison across multiple aspects        | Provides adequate comparison with some depth    | Limited comparison with few aspects        | Very minimal or no comparison            |
| Use of Evidence         | 20%           | Supports comparison with accurate data, examples, or references               | Uses some supporting evidence with minor gaps   | Limited or weak supporting evidence        | No supporting evidence provided          |
| Critical Thinking       | 20%           | Demonstrates strong critical thinking with clear interpretation and reasoning | Shows reasonable interpretation with minor gaps | Basic interpretation; lacks depth          | No meaningful interpretation             |
| Clarity of Presentation | 15%           | Well-organized, clear, and logically structured presentation                  | Generally clear with minor issues               | Somewhat unclear or poorly structured      | Disorganized and difficult to understand |

## ANSWERS

### Q1. Sequential File Organization vs. Heap File Organization

*Both are basic file organization techniques; they differ mainly in whether records are physically ordered by a key field.*

| Parameter       | Sequential File Organization                                                                                | Heap (Pile) File Organization                                            |
|-----------------|-------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Record order    | Physically sorted by a key field                                                                            | No particular order; records placed wherever space is free (append-only) |
| Insertion       | Slow – record must go into its correct sorted position, or into an overflow area merged later; up to $O(n)$ | Fast – new record is simply appended at the end; $O(1)$                  |
| Deletion        | Requires shifting/compacting records, or tombstone-marking followed by periodic reorganization              | Fast – record marked deleted; leaves fragmentation until reorganized     |
| Retrieval       | Efficient for ordered/range access; supports binary search, $O(\log n)$                                     | Inefficient – no order to exploit; needs a full linear scan, $O(n)$      |
| Best suited for | Batch processing, reporting, read- or range-heavy workloads                                                 | Write-heavy workloads, usually paired with a separate index for lookups  |

**Key Insight:** Sequential organization trades slower updates for fast ordered access, while heap organization trades slower retrieval for very fast, unordered insertion – the right choice depends on whether the workload is write-heavy or read/range-heavy.

### Q2. Clustered Indexing vs. Non-Clustered Indexing

*Clustering refers to whether the index order matches the physical storage order of the underlying data.*

| Parameter           | Clustered Index                                                    | Non-Clustered Index                                                      |
|---------------------|--------------------------------------------------------------------|--------------------------------------------------------------------------|
| Physical data order | Data rows are physically stored in the same order as the index key | Data rows are stored independently of the index order                    |
| Number per table    | Only one (data can be physically sorted only one way)              | Many can exist on the same table                                         |
| Structure           | Index and data are the same structure – the leaf level IS the data | A separate structure holding key values plus a pointer to the actual row |

| Parameter         | Clustered Index                                                                                                                                       | Non-Clustered Index                                                                                                                                       |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Range-query speed | Very fast – sequential physical access                                                                                                                | Slower – needs an extra lookup step (index → pointer → data page)                                                                                         |
| Example           | An Employees table with EmployeeID as primary key: rows are physically ordered by EmployeeID (default behaviour in engines such as SQL Server/InnoDB) | A separate index on the Department column of the same table: searching by Department scans the index, then follows the pointer to fetch each matching row |

**Key Insight:** A clustered index is best placed on the column most used for range scans or sorting (often the primary key); non-clustered indexes are then added on other frequently searched columns without disturbing the physical row order.

### Q3. Primary Indexing vs. Secondary Indexing

*These differ based on whether the index is built on the ordering key of a physically sorted file.*

| Parameter           | Primary Index                                                                    | Secondary Index                                                                                 |
|---------------------|----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Built on            | The ordering key of a file that is physically sorted on that key                 | A non-ordering field; the file need not be sorted on it                                         |
| Density             | Usually sparse – one entry per block, since the file is already sorted           | Must be dense – one entry per record, since matches are not physically clustered                |
| Storage requirement | Lower – index size is proportional to the number of blocks                       | Higher – index size is proportional to the number of records                                    |
| Search performance  | Very fast – small index, binary search plus a short scan within the target block | Slower than primary indexing due to larger index size, though still far faster than a full scan |

**Key Insight:** Primary indexes are storage-efficient because they piggyback on existing physical order, while secondary indexes trade extra storage for the flexibility of searching on fields other than the ordering key.

#### Q4. Static Hashing vs. Dynamic Hashing

*The two approaches differ in whether the number of buckets is fixed or allowed to change over time.*

| Parameter             | Static Hashing                                                                                  | Dynamic Hashing                                                                                         |
|-----------------------|-------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Number of buckets     | Fixed at file-creation time                                                                     | Grows or shrinks automatically as records are inserted or deleted                                       |
| Handling growth       | Records beyond capacity cause overflow, handled by overflow chaining, which lengthens over time | New buckets are created on demand (e.g. directory doubling in extendible hashing), avoiding long chains |
| Handling shrinkage    | Space is wasted if the file shrinks, since bucket count cannot reduce                           | Structure can shrink back and release unused space                                                      |
| Complexity            | Simple to implement and reason about                                                            | More complex – needs a directory or split pointer                                                       |
| Performance stability | Degrades as actual data size deviates from the original estimate                                | Stays stable across a wide range of database sizes                                                      |

**Key Insight:** Static hashing suits data whose size is known in advance and roughly stable; dynamic hashing is preferred when the database size fluctuates or grows unpredictably.

#### Q5. B-Tree Indexing vs. B+ Tree Indexing

*Both are balanced, multi-level tree structures; they differ mainly in where data pointers are stored.*

| Parameter                      | B-Tree                                                      | B+ Tree                                                                              |
|--------------------------------|-------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Data storage location          | Both internal and leaf nodes can hold record pointers       | Only leaf nodes hold record pointers; internal nodes store keys only, for navigation |
| Leaf-node linking              | Leaf nodes are not linked to one another                    | Leaf nodes are linked in a sequential list, enabling fast range scans                |
| Fan-out per node               | Lower – internal nodes also store data, using up node space | Higher – internal nodes store keys only, so more keys fit per node                   |
| Tree height (same data volume) | Tends to be taller – fewer keys per node                    | Tends to be shorter and broader – fewer disk I/Os                                    |

| Parameter          | B-Tree                                                                        | B+ Tree                                                                                         |
|--------------------|-------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Search consistency | A key may be found early at an internal node, giving inconsistent access time | Every search always goes to leaf level, giving predictable, uniform access time                 |
| Typical use        | Less common in commercial RDBMSs today                                        | Default indexing structure in most relational databases (e.g. MySQL InnoDB, PostgreSQL, Oracle) |

**Key Insight:** For large-scale applications with frequent range queries and ORDER BY-type operations, B+ Trees are almost always preferred over B-Trees because of their linked leaves and higher fan-out.

## Q6. Linear Hashing vs. Extendible Hashing

*Both are dynamic hashing schemes, but they differ in how and when buckets are split.*

| Parameter                  | Linear Hashing                                                                       | Extendible Hashing                                                                                  |
|----------------------------|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Directory                  | None needed – uses a single split pointer                                            | Requires a directory of $2^{(\text{global depth})}$ pointers to buckets                             |
| Which bucket splits        | Buckets split in a fixed, round-robin order, regardless of which one overflowed      | The specific bucket that overflows is split immediately                                             |
| Overflow before its "turn" | Handled with temporary overflow chaining until the split pointer reaches that bucket | Not an issue – the overflowing bucket is split right away                                           |
| Scalability                | Grows gradually, one bucket at a time; memory-efficient (no directory)               | Can double the directory instantly to relieve a hot bucket, but the directory itself can grow large |
| Complexity                 | Simpler – no directory maintenance                                                   | More complex – tracks local and global depth, and directory doubling/halving                        |

**Key Insight:** Linear hashing favours simplicity and low overhead at the cost of temporary overflow chains, while extendible hashing reacts immediately to a specific overflowing bucket at the cost of maintaining a directory.

## Q7. Dense Indexing vs. Sparse Indexing

*These differ in whether every record in the file receives its own index entry.*

| Parameter            | Dense Index                                            | Sparse Index                                                                          |
|----------------------|--------------------------------------------------------|---------------------------------------------------------------------------------------|
| Index entries        | One entry for every record in the file                 | One entry per block, typically for the block's first (anchor) record                  |
| Memory/storage usage | Higher – grows in proportion to the number of records  | Lower – grows in proportion to the number of blocks                                   |
| Applicability        | Works on both sorted and unsorted data files           | Only works when the data file is physically sorted on the indexed field               |
| Exact-lookup speed   | Faster – index gives the exact record pointer directly | Slightly slower – index gives the nearest block, then a short scan locates the record |

**Key Insight:** Dense indexes favour speed at the cost of storage; sparse indexes favour storage efficiency at a small, usually negligible search-time cost – sparse indexing is generally preferred whenever the file is already sorted.

## Q8. Indexed Sequential File Organization vs. Direct (Hashed) File Organization

*These are two strategies for combining fast access with practical file management.*

| Parameter     | Indexed Sequential (ISAM-style)                                                                                  | Direct / Hashed File Organization                                                                                       |
|---------------|------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Access method | Data kept in sorted order with an index for faster lookup; supports sequential scans and indexed random access   | Record address computed directly from the key via a hash function                                                       |
| Advantage     | Good balance – supports range queries/reporting and quick lookups                                                | Very fast exact-match retrieval, close to $O(1)$ average case                                                           |
| Limitation    | Index and overflow areas need periodic reorganization as the file grows, adding storage and maintenance overhead | Poor for range queries or sorted output, since hashing destroys key order; degrades with collisions or high load factor |

| Parameter       | Indexed Sequential (ISAM-style)                                                              | Direct / Hashed File Organization                                              |
|-----------------|----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Best suited for | Applications needing both batch and interactive access, e.g. payroll or banking master files | Applications dominated by exact-match lookups, e.g. key-based record retrieval |

**Key Insight:** ISAM offers flexibility across access patterns at the cost of maintenance overhead, while direct/hashed organization is extremely fast for point lookups but unsuitable when ordered or range-based access is also required.

### Q9. Single-Level Indexing vs. Multi-Level Indexing

*These differ in whether the index itself is further indexed, forming a hierarchy.*

| Parameter               | Single-Level Indexing                                                                                     | Multi-Level Indexing                                                                                    |
|-------------------------|-----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Structure               | One flat index mapping keys directly to block/record addresses                                            | A hierarchy of indexes – an index built on top of an index (the basis of B-Tree/B+ Tree structures)     |
| Behaviour as file grows | The index grows linearly with the file; once too large for memory, searching needs multiple disk accesses | Top level(s) stay small enough to fit in memory, since each level summarizes the one below it           |
| Search time             | Roughly $\log(n)$ via binary search, bounded by how many disk blocks the index occupies                   | Grows with the number of levels, which itself grows only logarithmically with data size – more scalable |
| Maintenance overhead    | Lower – only one structure to maintain                                                                    | Higher – insertions/deletions may propagate changes up multiple levels                                  |

**Key Insight:** Multi-level indexing is single-level indexing applied recursively, and becomes necessary once a flat index grows too large to search efficiently in memory – this same recursive idea underlies B-Tree and B+ Tree structures.



## Q10. Evaluating Hashing vs. Indexing for Exact-Match and Range Queries

*This question calls for a judgment, not just a description, so the two techniques are evaluated directly against the two query types that matter most in practice.*

| Query Type                               | Hashing                                                                                                    | Tree-based Indexing (e.g. B+ Tree)                                                                |
|------------------------------------------|------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Exact-match query                        | Excellent – average $O(1)$ , since the address is computed directly from the key                           | Good – $O(\log n)$ , requires traversing the tree from root to leaf                               |
| Range query<br>(BETWEEN, ORDER BY, etc.) | Poor – hashing scatters keys to distribute them evenly, so a range query degenerates into a near-full scan | Excellent – sorted structure with linked leaf nodes allows one seek followed by a sequential scan |
| Sensitivity to data distribution         | Can degrade with a poor hash function or high load factor (collisions)                                     | More predictable and stable, since the tree stays balanced                                        |

**Key Insight:** Neither technique is universally superior – the right choice depends on the dominant query pattern. Hashing should be favoured when the workload is dominated by exact-match lookups (e.g. primary-key retrieval in OLTP systems), since it gives near-constant time access. Indexing, particularly B+ Trees, should be favoured whenever range queries, sorting, or partial-match queries are common, since its sorted, linked-leaf structure directly supports sequential access. Most production databases combine both: hash indexes for equality lookups where supported, and B+ Tree indexes as the general-purpose default, since range queries are frequent enough in real applications that giving them up entirely would rarely be worth hashing's exact-match speed.

